

SOURCE CODE AND OUTPUT :

```
import pandas as pd
import numpy as np
data = pd.read_csv('data.csv')
print(f"Dataset successfully Imported of Shape : {data.shape}")
```

OUTPUT :

```
... Dataset successfully Imported of Shape : (76014, 15)
```

```
data.head()
```

OUTPUT :

	mid	date	venue	batting_team	bowling_team	batsman	bowler	runs	wickets	overs	runs_last_5	wickets_last_5	striker	non-striker	total
0	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	SC Ganguly	P Kumar	1	0	0.1	1	0	0	0	222
1	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	1	0	0.2	1	0	0	0	222
2	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.2	2	0	0	0	222
3	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.3	2	0	0	0	222
4	1	2008-04-18	M Chinnaswamy Stadium	Kolkata Knight Riders	Royal Challengers Bangalore	BB McCullum	P Kumar	2	0	0.4	2	0	0	0	222

```
data.describe()
```

OUTPUT :

	mid	runs	wickets	overs	runs_last_5	wickets_last_5	striker	non-striker	total
count	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000	76014.000000
mean	308.627740	74.889349	2.415844	9.783068	33.216434	1.120307	24.962283	8.869287	160.901452
std	178.156878	48.823327	2.015207	5.772587	14.914174	1.053343	20.079752	10.795742	29.246231
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	67.000000
25%	154.000000	34.000000	1.000000	4.600000	24.000000	0.000000	10.000000	1.000000	142.000000
50%	308.000000	70.000000	2.000000	9.600000	34.000000	1.000000	20.000000	5.000000	162.000000
75%	463.000000	111.000000	4.000000	14.600000	43.000000	2.000000	35.000000	13.000000	181.000000
max	617.000000	263.000000	10.000000	19.600000	113.000000	7.000000	175.000000	109.000000	263.000000

```
data.info()
```

OUTPUT :

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 76014 entries, 0 to 76013
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   mid                    76014 non-null  int64
1   date                   76014 non-null  object
2   venue                  76014 non-null  object
3   batting_team           76014 non-null  object
4   bowling_team           76014 non-null  object
5   batsman                76014 non-null  object
6   bowler                 76014 non-null  object
7   runs                   76014 non-null  int64
8   wickets                76014 non-null  int64
9   overs                  76014 non-null  float64
10  runs_last_5            76014 non-null  int64
11  wickets_last_5         76014 non-null  int64
12  striker                76014 non-null  int64
13  non-striker            76014 non-null  int64
14  total                  76014 non-null  int64
dtypes: float64(1), int64(8), object(6)
memory usage: 8.7+ MB

```

data.nunique()

OUTPUT :

```

mid                    617
date                   442
venue                   35
batting_team           14
bowling_team           14
batsman                411
bowler                 329
runs                   252
wickets                11
overs                  140
runs_last_5            102
wickets_last_5         8
striker                155
non-striker            88
total                  138
dtype: int64

```

data.dtypes

OUTPUT :

```

mid                    int64
date                   object
venue                  object
batting_team           object
bowling_team           object
batsman                object
bowler                 object
runs                   int64
wickets                int64
overs                  float64
runs_last_5            int64
wickets_last_5         int64
striker                int64
non-striker            int64
total                  int64
dtype: object

```

data.columns

OUTPUT :

```
Index(['mid', 'date', 'venue', 'batting_team', 'bowling_team', 'batsman',
      'bowler', 'runs', 'wickets', 'overs', 'runs_last_5', 'wickets_last_5',
      'striker', 'non-striker', 'total'],
      dtype='object')
```

```
irrelevant = ['mid', 'date', 'venue', 'batsman', 'bowler', 'striker', 'non-striker']
print(f'Before Removing Irrelevant Columns : {data.shape}')
data = data.drop(irrelevant, axis=1) # Drop Irrelevant Columns
print(f'After Removing Irrelevant Columns : {data.shape}')
data.head()
```

OUTPUT :

```
Before Removing Irrelevant Columns : (76014, 15)
After Removing Irrelevant Columns : (76014, 8)
```

	batting_team	bowling_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
0	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.1	1	0	222
1	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.2	1	0	222
2	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.2	2	0	222
3	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.3	2	0	222
4	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.4	2	0	222

```
const_teams = ['Kolkata Knight Riders', 'Chennai Super Kings', 'Rajasthan Royals',
               'Mumbai Indians', 'Kings XI Punjab', 'Royal Challengers Bangalore',
               'Delhi Daredevils', 'Sunrisers Hyderabad']
print(f'Before Removing Inconsistent Teams : {data.shape}')
data=data[(data['batting_team'].isin(const_teams))&(data['bowling_team'].isin(const_teams))]
print(f'After Removing Irrelevant Columns : {data.shape}')
print(f"Consistent Teams : \n{data['batting_team'].unique()}")
data.head()
```

OUTPUT :

```
Before Removing Inconsistent Teams : (76014, 8)
After Removing Irrelevant Columns : (53811, 8)
Consistent Teams :
['Kolkata Knight Riders' 'Chennai Super Kings' 'Rajasthan Royals'
 'Mumbai Indians' 'Kings XI Punjab' 'Royal Challengers Bangalore'
 'Delhi Daredevils' 'Sunrisers Hyderabad']
```

	batting_team	bowling_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
0	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.1	1	0	222
1	Kolkata Knight Riders	Royal Challengers Bangalore	1	0	0.2	1	0	222
2	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.2	2	0	222
3	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.3	2	0	222
4	Kolkata Knight Riders	Royal Challengers Bangalore	2	0	0.4	2	0	222

```
print(f'Before Removing Overs : {data.shape}')
data = data[data['overs'] >= 5.0]
print(f'After Removing Overs : {data.shape}')
```

```
data.head()
```

OUTPUT :

```
Before Removing Overs : (53811, 8)
After Removing Overs : (40108, 8)
```

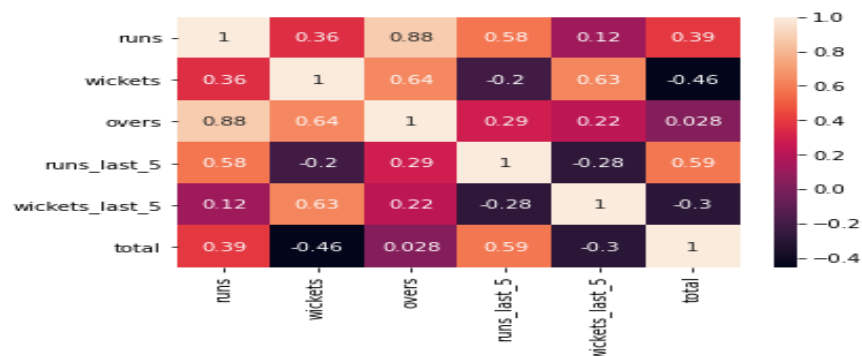
	batting_team	bowling_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
32	Kolkata Knight Riders	Royal Challengers Bangalore	61	0	5.1	59	0	222
33	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.2	59	1	222
34	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.3	59	1	222
35	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.4	59	1	222
36	Kolkata Knight Riders	Royal Challengers Bangalore	61	1	5.5	58	1	222

```
from seaborn import heatmap
```

```
heatmap(data=data.corr(), annot=True)
```

OUTPUT :

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fc89bb99ef0>
```



```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
le = LabelEncoder()
```

```
for col in ['batting_team', 'bowling_team']:
```

```
    data[col] = le.fit_transform(data[col])
```

```
data.head()
```

OUTPUT :

	batting_team	bowling_team	runs	wickets	overs	runs_last_5	wickets_last_5	total
32	3	6	61	0	5.1	59	0	222
33	3	6	61	1	5.2	59	1	222
34	3	6	61	1	5.3	59	1	222
35	3	6	61	1	5.4	59	1	222
36	3	6	61	1	5.5	58	1	222

```
from sklearn.compose import ColumnTransformer
```

```
columnTransformer = ColumnTransformer([('encoder',
```

```
    OneHotEncoder(),
```

```
    [0, 1])),
```

```
    remainder='passthrough')
```

```
data = np.array(columnTransformer.fit_transform(data))
```

```

cols = ['batting_team_Chennai Super Kings', 'batting_team_Delhi Daredevils',
'batting_team_Kings XI Punjab',
        'batting_team_Kolkata Knight Riders', 'batting_team_Mumbai Indians',
'batting_team_Rajasthan Royals',
        'batting_team_Royal Challengers Bangalore', 'batting_team_Sunrisers Hyderabad',
        'bowling_team_Chennai Super Kings', 'bowling_team_Delhi Daredevils',
'bowling_team_Kings XI Punjab',
        'bowling_team_Kolkata Knight Riders', 'bowling_team_Mumbai Indians',
'bowling_team_Rajasthan Royals',
        'bowling_team_Royal Challengers Bangalore', 'bowling_team_Sunrisers Hyderabad',
'runs', 'wickets', 'overs',
        'runs_last_5', 'wickets_last_5', 'total']
df = pd.DataFrame(data, columns=cols)
df.head()

```

OUTPUT :

	batting_team_Chennai Super Kings	batting_team_Delhi Daredevils	batting_team_Kings XI Punjab	batting_team_Kolkata Knight Riders	batting_team_Mumbai Indians	batting_team_Rajasthan Royals	batting_team_Royal Challengers Bangalore	batting_team_Sunri Hyders
0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	0.0	0.0	0.0	

	bowling_team_Kolkata Knight Riders	bowling_team_Mumbai Indians	bowling_team_Rajasthan Royals	bowling_team_Royal Challengers Bangalore	bowling_team_Sunrisers Hyderabad	runs	wickets	overs	runs_last_5	wickets_last_5	total
	0.0	0.0	0.0	1.0	0.0	61.0	0.0	5.1	59.0	0.0	222.0
	0.0	0.0	0.0	1.0	0.0	61.0	1.0	5.2	59.0	1.0	222.0
	0.0	0.0	0.0	1.0	0.0	61.0	1.0	5.3	59.0	1.0	222.0
	0.0	0.0	0.0	1.0	0.0	61.0	1.0	5.4	59.0	1.0	222.0
	0.0	0.0	0.0	1.0	0.0	61.0	1.0	5.5	58.0	1.0	222.0

```

features = df.drop(['total'], axis=1)
labels = df['total']
# Perform 80 : 20 Train-Test split
from sklearn.model_selection import train_test_split
train_features, test_features, train_labels, test_labels = train_test_split(features, labels,
test_size=0.20, shuffle=True)
print(f"Training Set : {train_features.shape}\nTesting Set : {test_features.shape}")

```

OUTPUT :

```

Training Set : (32086, 21)
Testing Set : (8022, 21)

```

```

models = dict()
from sklearn.linear_model import LinearRegression

```

```

linreg = LinearRegression()
linreg.fit(train_features, train_labels)
train_score_linreg = str(linreg.score(train_features, train_labels) * 100)
test_score_linreg = str(linreg.score(test_features, test_labels) * 100)
print(f'Train Score : {train_score_linreg[:5]}%\nTest Score : {test_score_linreg[:5]}%')
models["linreg"] = test_score_linreg

```

OUTPUT :

```

Training Set : (32086, 21)
Testing Set : (8022, 21)

```

```

print("---- Linear Regression - Model Evaluation ----")
print("Mean Absolute Error(MAE):{}".format(mae(test_labels,linreg.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels,linreg.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels,
linreg.predict(test_features)))))

```

OUTPUT :

```

---- Linear Regression - Model Evaluation ----
Mean Absolute Error (MAE): 13.095521348645773
Mean Squared Error (MSE): 301.4403800998092
Root Mean Squared Error (RMSE): 17.36203847766181

```

```

from sklearn.ensemble import RandomForestRegressor
forest = RandomForestRegressor()
forest.fit(train_features, train_labels)
# Evaluate Model
train_score_forest = str(forest.score(train_features, train_labels)*100)
test_score_forest = str(forest.score(test_features, test_labels)*100)
print(f'Train Score : {train_score_forest[:5]}%\nTest Score : {test_score_forest[:5]}%')
models["forest"] = test_score_forest

```

OUTPUT :

```

Train Score : 99.07%
Test Score : 93.08%

```

```

print("---- Random Forest Regression - Model Evaluation ----")
print("Mean Absolute Error(MAE):{}".format(mae(test_labels,forest.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels,forest.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels,
forest.predict(test_features)))))

```

OUTPUT :

```
---- Random Forest Regression - Model Evaluation ----
Mean Absolute Error (MAE): 4.608180013890372
Mean Squared Error (MSE): 61.16125239745949
Root Mean Squared Error (RMSE): 7.820565989585376
```

```
from sklearn.neural_network import MLPRegressor
neural_net = MLPRegressor(activation='logistic', max_iter=500)
neural_net.fit(train_features, train_labels)
train_score_neural_net = str(neural_net.score(train_features, train_labels)*100)
test_score_neural_net = str(neural_net.score(test_features, test_labels)*100)
print(f'Train Score : {train_score_neural_net[:5]}%\nTest Score : {test_score_neural_net[:5]}%')
models["neural_net"] = test_score_neural_net
```

OUTPUT :

```
Train Score : 86.27%
Test Score : 84.68%
```

```
print("---- Neural Networks Regression - Model Evaluation ----")
print("Mean Absolute Error (MAE): {}".format(mae(test_labels,
neural_net.predict(test_features))))
print("Mean Squared Error (MSE): {}".format(mse(test_labels,
neural_net.predict(test_features))))
print("Root Mean Squared Error (RMSE): {}".format(np.sqrt(mse(test_labels,
neural_net.predict(test_features)))))
```

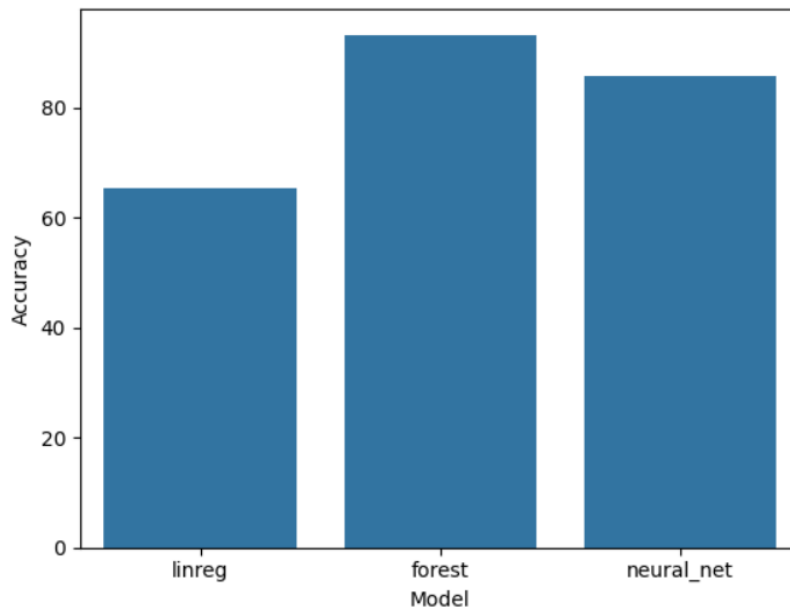
OUTPUT :

```
---- Neural Networks Regression - Model Evaluation ----
Mean Absolute Error (MAE): 8.252417080313622
Mean Squared Error (MSE): 135.47139255624822
Root Mean Squared Error (RMSE): 11.639217867032485
```

```
from seaborn import barplot
model_names = list(models.keys())
accuracy = list(map(float, models.values()))
barplot(model_names, accuracy)
```

OUTPUT :

```
[37]: <Axes: xlabel='Model', ylabel='Accuracy'>
```



```
def predict_score(batting_team, bowling_team, runs, wickets, overs, runs_last_5,
wickets_last_5, model=forest):
    prediction_array = []
    if batting_team == 'Chennai Super Kings':
        prediction_array = prediction_array + [1,0,0,0,0,0,0,0]
    elif batting_team == 'Delhi Daredevils':
        prediction_array = prediction_array + [0,1,0,0,0,0,0,0]
    elif batting_team == 'Kings XI Punjab':
        prediction_array = prediction_array + [0,0,1,0,0,0,0,0]
    elif batting_team == 'Kolkata Knight Riders':
        prediction_array = prediction_array + [0,0,0,1,0,0,0,0]
    elif batting_team == 'Mumbai Indians':
        prediction_array = prediction_array + [0,0,0,0,1,0,0,0]
    elif batting_team == 'Rajasthan Royals':
        prediction_array = prediction_array + [0,0,0,0,0,1,0,0]
    elif batting_team == 'Royal Challengers Bangalore':
        prediction_array = prediction_array + [0,0,0,0,0,0,1,0]
    elif batting_team == 'Sunrisers Hyderabad':
        prediction_array = prediction_array + [0,0,0,0,0,0,0,1]
    if bowling_team == 'Chennai Super Kings':
        prediction_array = prediction_array + [1,0,0,0,0,0,0,0]
    elif bowling_team == 'Delhi Daredevils':
        prediction_array = prediction_array + [0,1,0,0,0,0,0,0]
    elif bowling_team == 'Kings XI Punjab':
        prediction_array = prediction_array + [0,0,1,0,0,0,0,0]
```



```

elif bowling_team == 'Kolkata Knight Riders':
    prediction_array = prediction_array + [0,0,0,1,0,0,0,0]
elif bowling_team == 'Mumbai Indians':
    prediction_array = prediction_array + [0,0,0,0,1,0,0,0]
elif bowling_team == 'Rajasthan Royals':
    prediction_array = prediction_array + [0,0,0,0,0,1,0,0]
elif bowling_team == 'Royal Challengers Bangalore':
    prediction_array = prediction_array + [0,0,0,0,0,0,1,0]
elif bowling_team == 'Sunrisers Hyderabad':
    prediction_array = prediction_array + [0,0,0,0,0,0,0,1]
prediction_array = prediction_array + [runs, wickets, overs, runs_last_5, wickets_last_5]
prediction_array = np.array([prediction_array])
pred = model.predict(prediction_array)
return int(round(pred[0]))

```

Live Test 1

```

batting_team="Kings XI Punjab"
bowling_team="Rajasthan Royals"
score = predict_score(batting_team, bowling_team, overs=14.0, runs=118, wickets=1,
runs_last_5=45, wickets_last_5=0)
print(f'Predicted Score : {score}')

```

OUTPUT :

Predicted Score : 178

Live Test 2

```

batting_team="Kolkata Knight Riders"
bowling_team="Chennai Super Kings"
score = predict_score(batting_team, bowling_team, overs=18.0, runs=150, wickets=4,
runs_last_5=57, wickets_last_5=1)
print(f'Predicted Score : {score} ')

```

OUTPUT :

Predicted Score : 175

Live Test 3

```

batting_team='Delhi Daredevils'
bowling_team='Mumbai Indians'
score = predict_score(batting_team, bowling_team, overs=18.0, runs=96, wickets=8,
runs_last_5=18, wickets_last_5=4)
print(f'Predicted Score : {score}')
```

OUTPUT :

Predicted Score : 107 |